

LIPv4: a learned iterative planner for latent world models

Learned Iterative Planning, “min0” variant — goal-reaching in a frozen world model. Internal note, 2026-07-20.

LIPv4 replaces sampling-based CEM planning with a *learned* refinement rule. A small MLP f_θ (two hidden layers, width 512) is applied K times, weight-tied, to an action plan $A \in \mathbb{R}^{H \times a}$ ($H=5$ blocks of $a=25$ dims = 5 primitives $\times$ 5-d action):

$$A^{(0)} = 0, \quad A^{(k+1)} = \underbrace{\text{clip}_{\pm a_{\max}}}_{\text{clip}} \left(\underbrace{A^{(k)} + f_\theta(A^{(k)}, \nabla_A V, V)}_{\text{residual update}} \right), \quad k = 0, \dots, K-1.$$

Here $V = V_\phi(\hat{z}_H, z_g)$ is a learned, goal-conditioned terminal value evaluated on the plan’s imagined final latent $\hat{z}_H$ — obtained by rolling the *frozen* world model forward under A — and $\nabla_A V$ is its gradient back through the world model. The v4 (“min0”) form is **input-minimal and gate-free**: the actor sees only $[A, \nabla_A V, V]$ — no raw state, no raw goal. Task information reaches the planner *only* through the value.

Clipped residual (our v4 form). f_θ emits a *residual* Δ , not an absolute plan, so each of the K weight-tied iterations refines the previous one and every iterate is hard-clipped to $\pm a_{\max}$: the update uses *both* the residual (incremental, near-identity at init) *and* the clip (bounds the plan to the training action range). v4 drops the sigmoid gate of earlier versions — the clip takes over its damping role, so “gate-free” is not “unbounded.” Deploy is receding-horizon: run K iterations from $A^{(0)}=0$, execute the first block, re-plan.

Critic. V_ϕ is a goal-conditioned temporal-distance value (quasimetric head) trained by n -step TD ($n=50$) with hindsight (HER) goals and a low expectile ($\tau=0.03$, optimistic toward the min), warm-started from a sequential-TD checkpoint and co-trained 1:1 with the actor through an EMA teacher.

Results — OGBench cube, LeWM world model (3 eval draws $\times$ 50 tasks)

planner (success rate, %)	h25	h50
random floor (tasks already solved at start)	50.7	30.7
native CEM (world model’s own latent-MSE cost)	75.3	54.0
TD+CEM (learned value, CEM search)	79.3	64.7
LIPv4 (learned value + learned optimizer)	87.6	74.0

The architecture transfers *unchanged* to a different world model (PLDM): +12.3 at h25, +7.0 at h50 over base PLDM+CEM, with the largest relative gain exactly where the world model’s native cost is weakest (over-floor ratio $1.93\times$ vs. CEM on PLDM at h25, vs. $1.51\times$ on LeWM).

Why it works

The value-gradient is the load-bearing input. Ablating it ($[A, \nabla_A V, V] \rightarrow [A, V]$) collapses success $87.6 \rightarrow 54.0$ — the gradient carries $\approx 91\%$ of all above-floor performance. A scalar value cannot steer a 125-d action space; the gradient supplies direction (CEM substitutes brute-force sampling for it).

$\sim 500\times$ cheaper planning. ≈ 16 world-model rollout-equivalents per replan versus $\approx 9,000$ for CEM — at *higher* success.

Less input is more. Dropping raw state, raw goal, and the gate ties or beats the full-input actor at lower variance: the value is the only teacher, and the only input.

Limits

About 12 “grasp-with-airborne-goal” tasks fail for *every* planner (LIPv4 and CEM alike): a property of expert-only data — zero attempted-and-missed grasps in 2M frames — fixable only by a negative-data world-model fine-tune (89.3, breaking the long-standing 88 plateau), not by any planner-side change. The gradient refiner is also more sensitive than CEM to off-manifold states at the receding-horizon replan boundary.

Recipe: $K=8$, $a_{\max}=3.5\sigma$, actor lr 3×10^{-4} , critic $n=50/\tau=0.03$, $A^{(0)}=0$, 1 restart. Code: `stable_worldmodel/solver/lip.py` (`kind='lip4'`); trainer `scripts/plan/train_lip_ac.py`.